

Purpose

This document explains how the GC-Game system fits together operationally. It is designed to re-orient you after time away and clarify what runs where, what is safe to change, and what is not.

System Topology

- Backend: FastAPI running in Docker
- Database: PostgreSQL in Docker
- Dev Hosts: Windows (WSL2) and macOS (Docker Desktop)
- Production Host: Raspberry Pi 5 (Docker only)
- Client: Godot desktop app (distributed separately)

Core Principle

Docker is the source of truth everywhere. No local Python virtualenvs are used for runtime. Local environments exist only to build, inspect, or test containers.

Dev vs Production Rules

DEV (Windows/Mac):

- Containers run with `--reload`
- Frequent rebuilds are expected
- Database resets are acceptable

PROD (Pi):

- No `--reload`
- Containers restart automatically
- Database persistence is critical

Deployment Flow

1. Develop on Windows or Mac
2. Commit and push to GitHub
3. SSH into Pi
4. `git pull`
5. `docker compose up -d --build`

What Is Safe to Break

- Local containers on dev machines
- Alembic revisions before first public game
- Local databases during development

What Is NOT Safe to Break

- Production PostgreSQL volume
- Turn resolution integrity
- User authentication tables

Failure Scenarios

- Pi reboot: Docker restarts containers automatically
- Network drop: Game pauses until connectivity restored
- Player disconnect: No impact on turn-based model

When to Touch the Pi

- Deploying milestones
 - Verifying production behavior
 - Checking logs after issues
- Do NOT develop directly on the Pi.

Mental Checklist

- Am I on dev or prod?
- Should this rebuild destroy data?
- Did I commit before pulling on Pi?
- Is --reload enabled only where it should be?